



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TLÁHUAC

“La esencia de la grandeza radica en las raíces”

Desarrollo Backend

Manual de Usuario de la BD (Postgres)

Docente: Ing. Jaqueline Padilla Juarez

Alumnos:

Chavarria Reyes Valeria

Cisneros Garcia Tifany Alexandra

De la Rosa Jimenez Denilson Djorkaef

Grupo: 8S1



Índice

Introducción.....	3
Objetivos.....	3
Requerimientos del sistema	3
Instalación de Postgres	4
Diagrama entidad - relación	8
Estructura de la Base de Datos “Condon”	8
Funcionamiento de la Base de Datos “Condon”	11
Consultas.....	11
Conexión de la Base de Datos con el Proyecto.....	12
Configuración de dependencias	12
Variables de entorno	12
Módulo de conexión	13
Conclusión	13

Introducción

La base de datos de nuestro proyecto (El mundo del condon 2.0), la cual lleva por nombre “**condon**” fue creada con el propósito de simular un sistema de **e-commerce especializado en productos de salud sexual**, en el cual los usuarios pueden registrarse, navegar por un catálogo de productos, gestionar un carrito de compras, realizar pedidos y efectuar pagos.

Este manual de usuario está orientado hacia aquellos que necesiten comprender el diseño de la base de datos, sus funciones principales y el uso de consultas SQL para extraer y manipular la información almacenada.

Objetivos

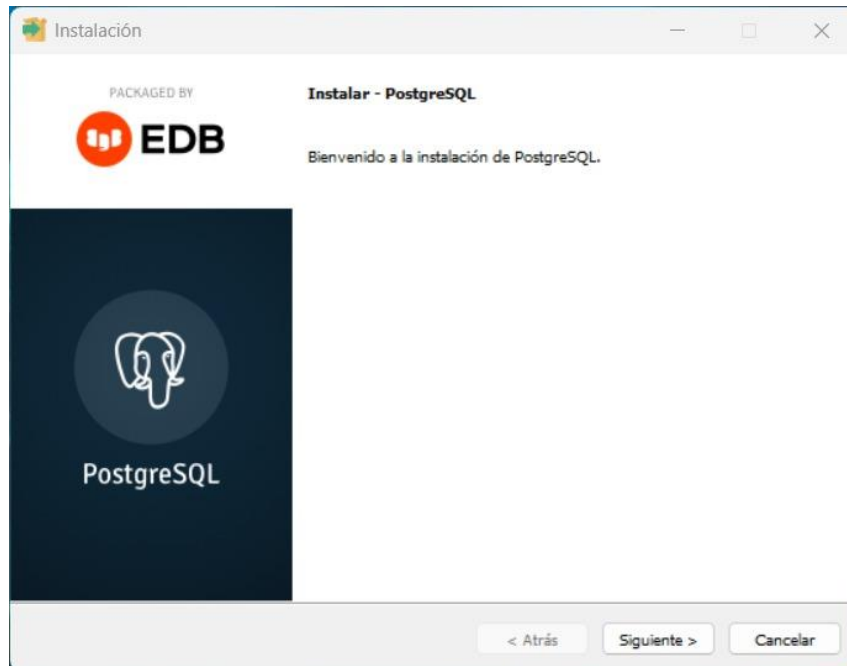
- Centralizar la información de usuarios, proveedores, productos, pedidos y pagos.
- Mantener la **integridad referencial** de los datos mediante claves primarias y foráneas.
- Automatizar procesos como el cálculo del **total de un pedido** mediante **triggers**.
- Permitir un crecimiento escalable a futuro, soportando más usuarios y productos sin comprometer el rendimiento.
- Servir como soporte para aplicaciones web desarrolladas en **Astro + Node + PostgreSQL**.

Requerimientos del sistema

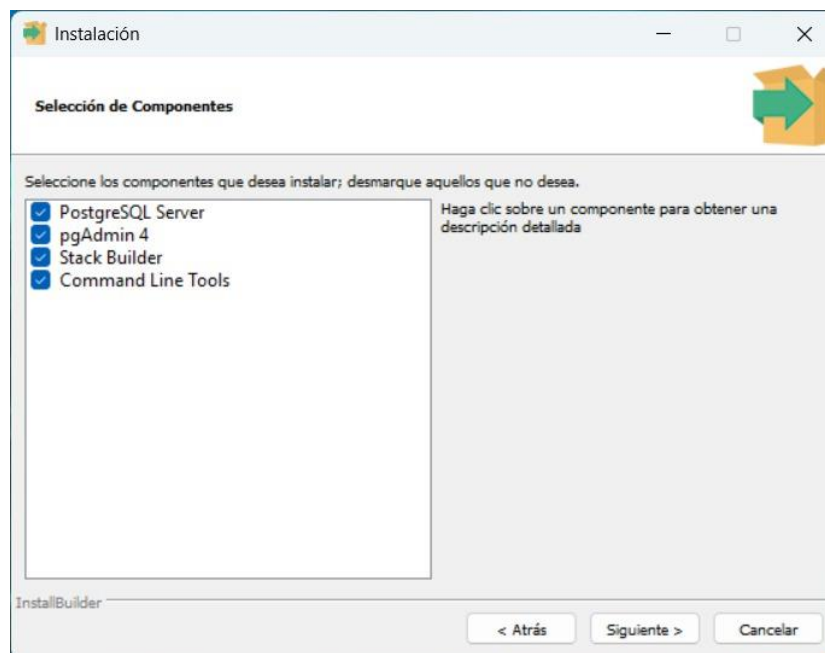
- Motor de base de datos: PostgreSQL 15+
- Herramienta de administración: pgAdmin 4
- Lenguaje de consultas: SQL estándar
- Integración: Framework Astro (Node.js), librería pg

Instalación de Postgres

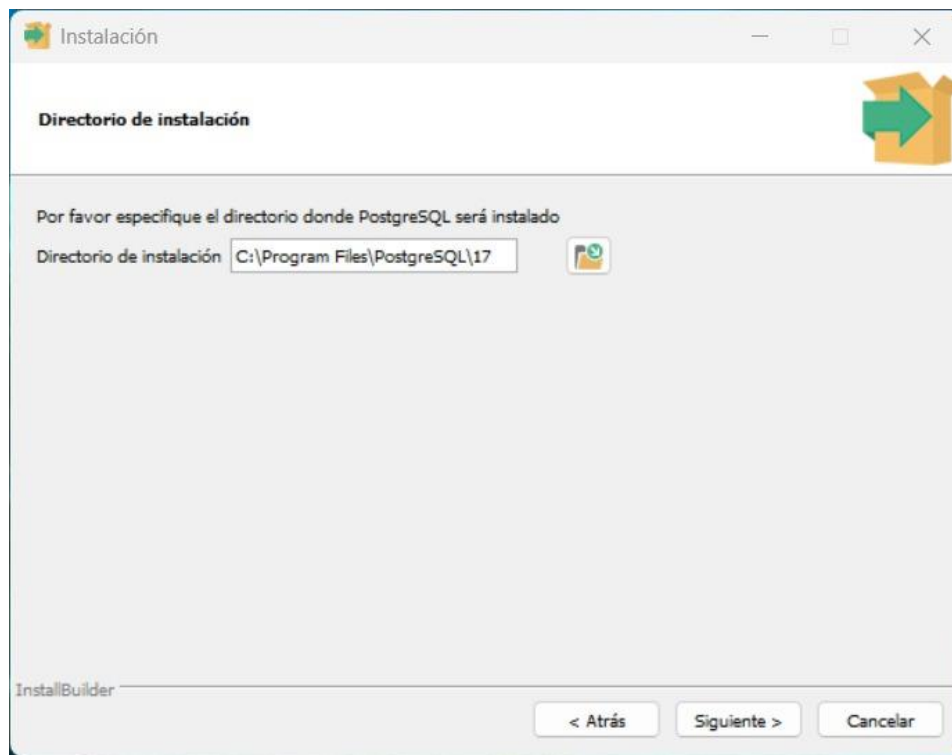
1. Lo primero que tenemos que hacer es realizar la instalación de postgres desde el siguiente link: <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>
2. Después abriremos el launcher y comenzaremos con la instalación



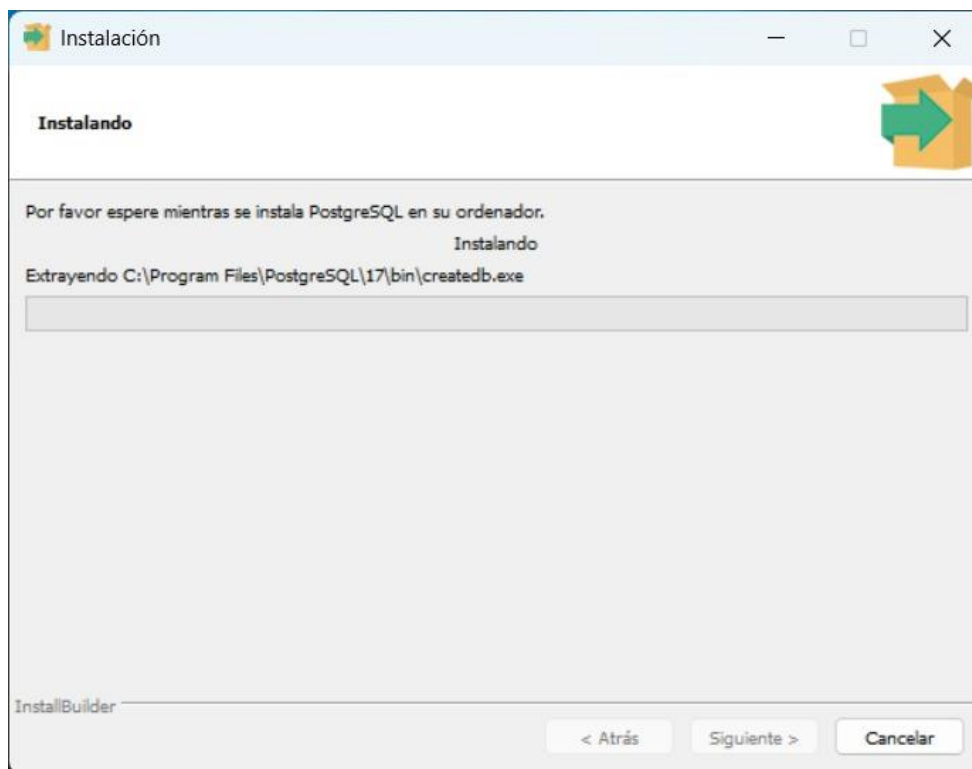
3. Seleccionaremos los componentes a instalar



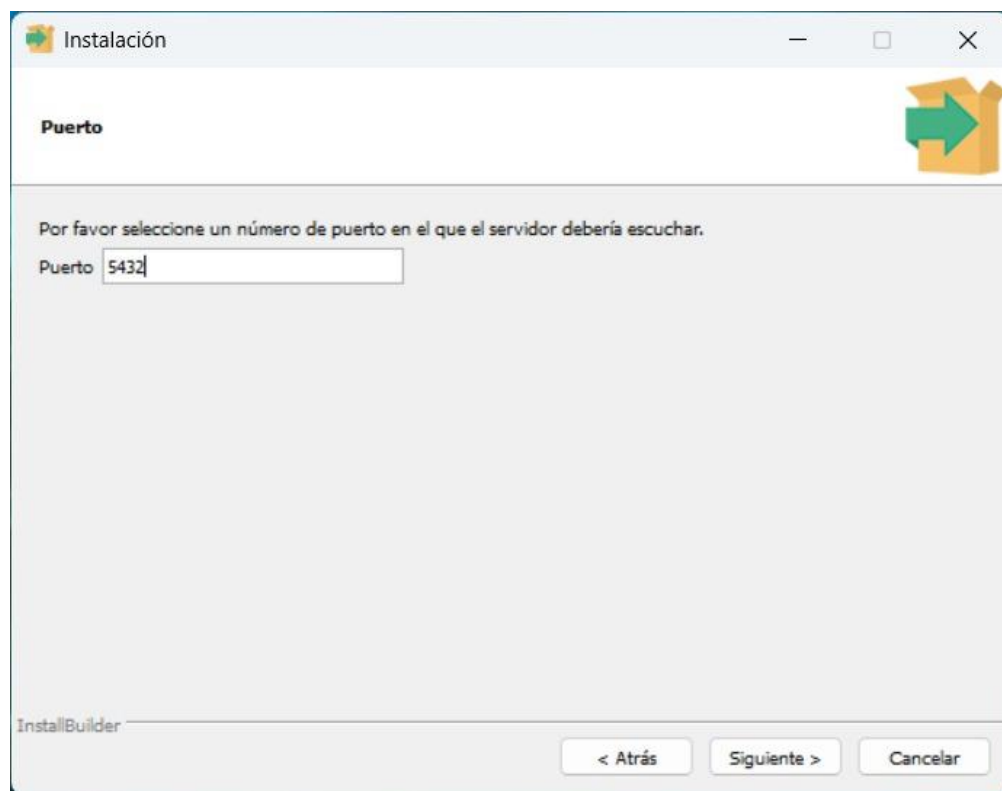
4. Especificaremos en donde queremos hacer la instalación



4. Comenzará la instalación

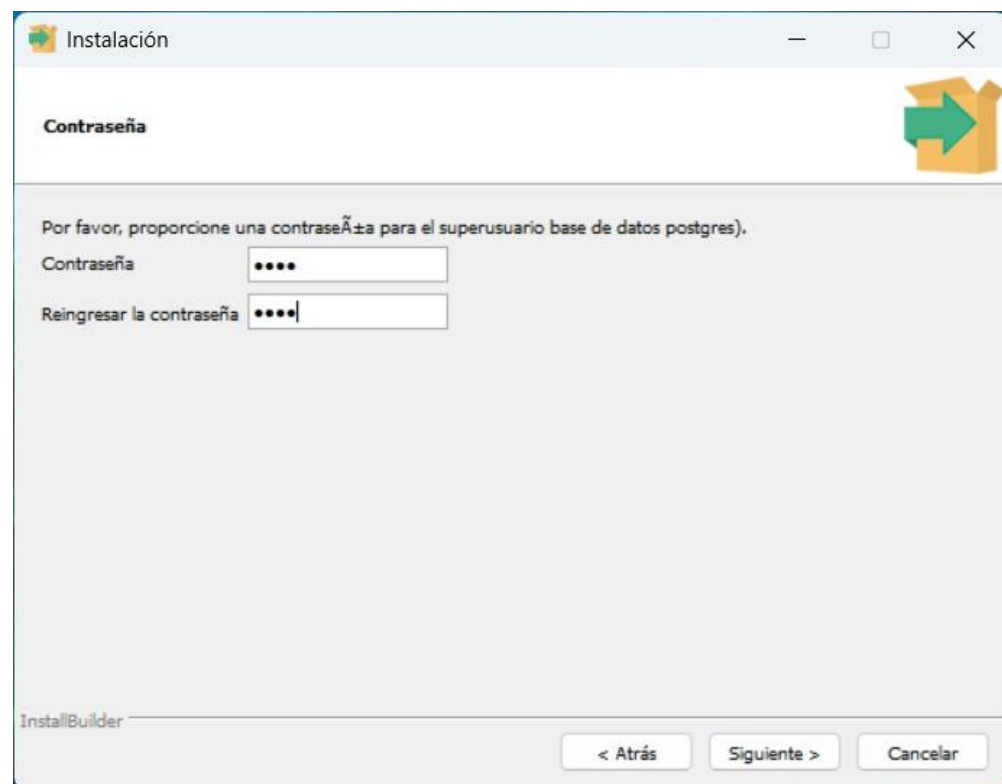


5. Elegiremos el puerto que usara postgres



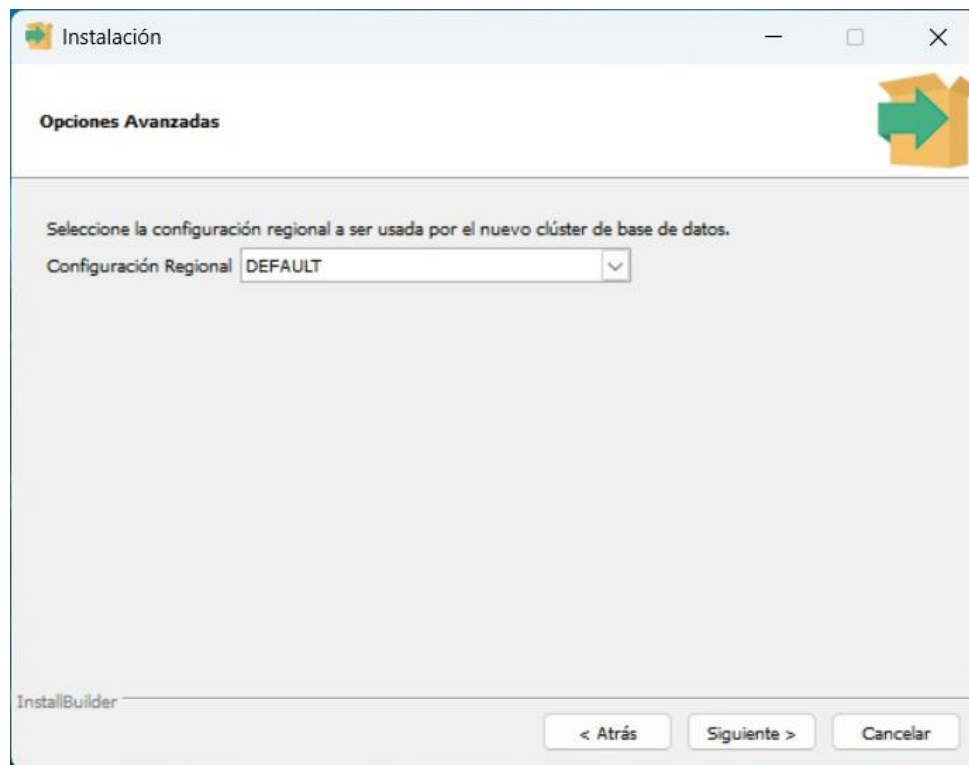
The screenshot shows a window titled "Instalación" with a standard Windows title bar. The main content area is titled "Puerto" and contains the instruction: "Por favor seleccione un número de puerto en el que el servidor debería escuchar." Below this, there is a text input field labeled "Puerto" containing the value "5432". In the bottom right corner, there are three buttons: "< Atrás", "Siguiete >", and "Cancelar". The bottom left corner of the window displays the text "InstallBuilder". A green arrow icon pointing right is located in the top right corner of the main content area.

6. Asignaremos una contraseña



The screenshot shows the same "Instalación" window, but now the main content area is titled "Contraseña". The instruction reads: "Por favor, proporcione una contraseña para el superusuario base de datos postgres)." There are two text input fields: the first is labeled "Contraseña" and contains four dots, and the second is labeled "Reingresar la contraseña" and also contains four dots. The same three buttons ("< Atrás", "Siguiete >", "Cancelar") are in the bottom right, and "InstallBuilder" is in the bottom left. The green arrow icon remains in the top right corner of the main content area.

7. Seleccionaremos la configuración regional



8. Finalmente estará instalado el postgres

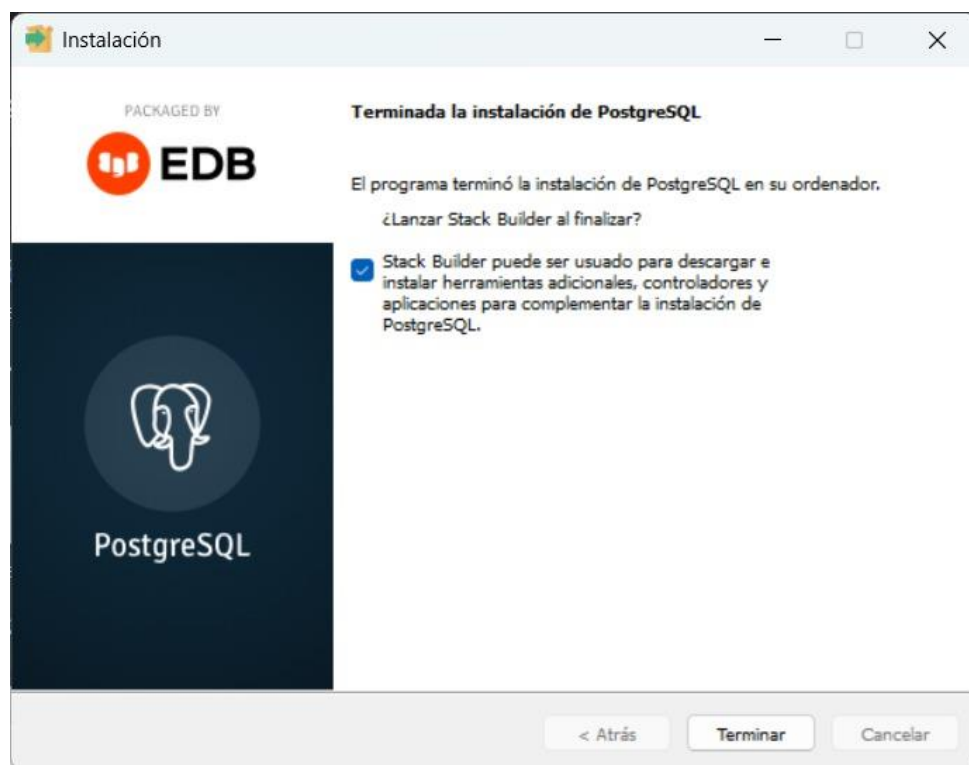
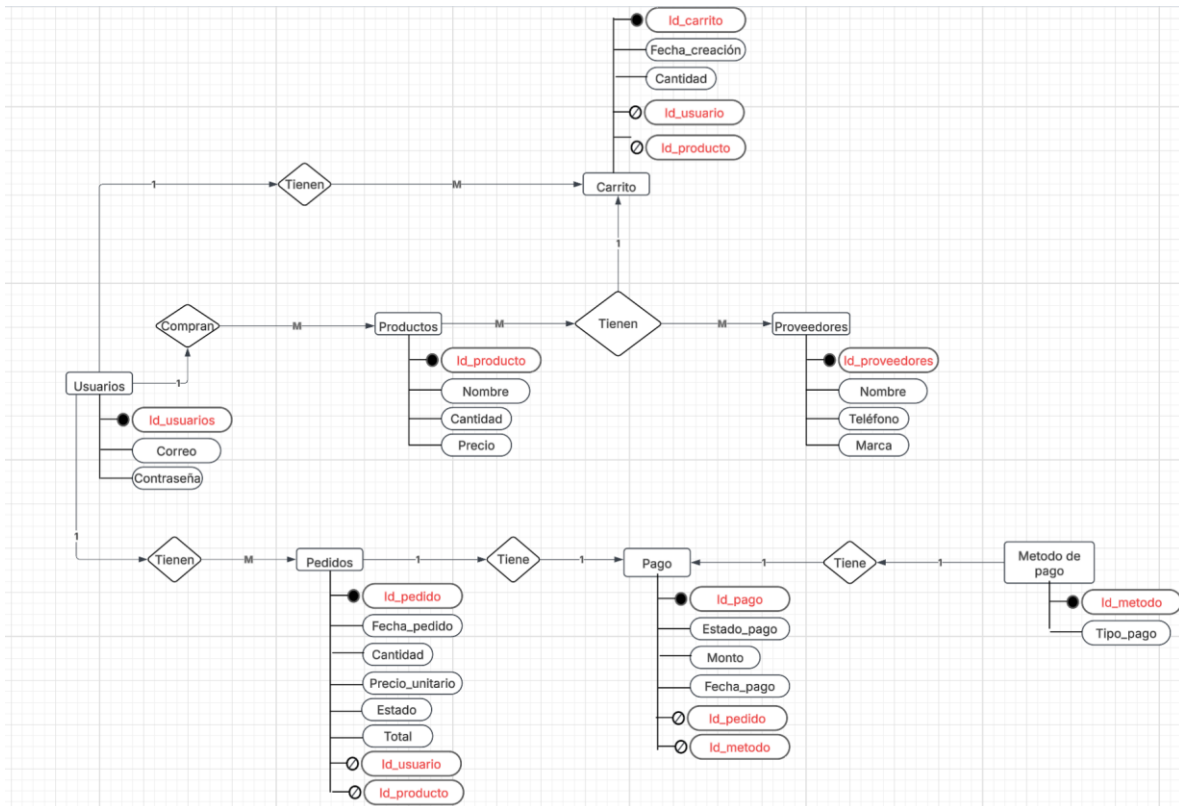


Diagrama entidad- relación



Estructura de la Base de Datos "Condon"

La BD está compuesta por las siguientes **tablas principales**:

1. Usuarios

Campo	Tipo	Descripción
id_usuario	SERIAL PK	Identificador único de usuario
correo	VARCHAR	Correo electrónico (único)
contrasena	VARCHAR	Contraseña del usuario
creado_en	TIMESTAMP	Fecha de registro

2. Proveedores

Campo	Tipo	Descripción
id_proveedor	SERIAL PK	Identificador del proveedor
nombre	VARCHAR	Nombre de la empresa
telefono	VARCHAR	Teléfono de contacto
marca	VARCHAR	Marca de productos

3. Productos

Campo	Tipo	Descripción
id_producto	SERIAL PK	Identificador del producto
nombre	VARCHAR	Nombre del producto
precio	NUMERIC	Precio unitario
cantidad	INTEGER	Stock disponible
id_proveedor	INT FK	Relación con tabla proveedores

4. Carritos

Campo	Tipo	Descripción
id_carrito	SERIAL PK	Identificador del carrito
id_usuario	INT FK	Usuario dueño del carrito
fecha_creacion	TIMESTAMP	Fecha en que se creó el carrito

5. Carrito_items

Campo	Tipo	Descripción
id_carrito	INT FK	Carrito asociado
id_producto	INT FK	Producto agregado
cantidad	INT	Cantidad seleccionada

6. Pedidos

Campo	Tipo	Descripción
id_pedido	SERIAL PK	Identificador del pedido
id_usuario	INT FK	Usuario que realizó el pedido
fecha_pedido	TIMESTAMP	Fecha del pedido
estado	VARCHAR	Estado del pedido (pendiente, pagado, enviado)
total	NUMERIC	Total calculado (por trigger)

7. Pedido_items

Campo	Tipo	Descripción
id_pedido	INT FK	Pedido asociado
id_producto	INT FK	Producto comprado
cantidad	INT	Cantidad comprada
precio_unit	NUMERIC	Precio unitario al momento

8. Pagos

Campo	Tipo	Descripción
id_pago	SERIAL PK	Identificador del pago
id_pedido	INT FK	Pedido asociado
id_metodo	INT FK	Método de pago utilizado
monto	NUMERIC	Cantidad abonada
estado_pago	VARCHAR	Estado (pagado, pendiente, fallido)
fecha_pago	TIMESTAMP	Fecha del pago

9. Metodos_pago

Campo	Tipo	Descripción
id_metodo	SERIAL PK	Identificador del método
tipo_pago	VARCHAR	Tipo (tarjeta, efectivo)

Funcionamiento de la Base de Datos “Condon”

1. Registro de usuarios: cada usuario se guarda en usuarios con correo y contraseña.
2. Proveedores y productos: los proveedores se registran en proveedores y se relacionan con productos en productos.
3. Carrito de compras: al navegar, los usuarios llenan un carrito (carritos) y sus detalles (carrito_items).
4. Pedido: al confirmar, se crea un pedido con sus pedido_items, calculando automáticamente el total.
5. Pago: los pagos se almacenan en pagos y se ligam a metodos_pago.

Consultas

Método POST

```
const sql = `INSERT INTO usuarios (correo, contraseña)
VALUES ($1,$2) RETURNING id_usuario, correo, creado_en`;
```

Método GET

	id_usuario [PK] bigint	correo character varying (255)	contrasena character varying (255)	creado_en timestamp without time zone
1	13	fany@gmail.com	1234	2025-09-11 07:18:11.021926
2	14	deni1@gmail.com	4321	2025-09-11 08:32:32.587953

Método PUT

```
export const PUT: APIRoute = async ({ params, request }) => {
  const id = Number(params.id);
  const { correo, contraseña } = await request.json();
  await query(
    'UPDATE usuarios SET correo=$1, contraseña=$2 WHERE id_usuario=$3;',
    [correo, contraseña, id]
  );
  return new Response(JSON.stringify({ ok: true }));
};
```

Método DELETE

```
export const DELETE: APIRoute = async ({ params, url }) => {
  const id_producto = Number(params.id);
  const id_usuario = Number(url.searchParams.get('user'));
  if (!id_usuario || !id_producto) return new Response(JSON.stringify({ error: 'user e id del producto requeridos' }), { status: 400 });
```

Método PUSH

```
await client.query(`
  INSERT INTO carrito_items (id_carrito, id_producto, cantidad)
  VALUES ($1,$2,$3)
  ON CONFLICT (id_carrito, id_producto)
  DO UPDATE SET cantidad = carrito_items.cantidad + EXCLUDED.cantidad;`,
  [id_carrito, id_producto, cantidad]
);
```

Conexión de la Base de Datos con el Proyecto

La base de datos **condon** se integró en un proyecto web desarrollado con **Astro**, utilizando **Node.js** en el servidor y la librería **pg** para interactuar con PostgreSQL.

Configuración de dependencias

En el archivo package.JSON agregamos las dependencias necesarias “dotenv” y “pg”

```
"dependencies": {
  "@astrojs/react": "^4.3.0",
  "@types/react": "^19.1.5",
  "@types/react-dom": "^19.1.5",
  "astro": "^5.7.13",
  "dotenv": "^17.2.2",
  "framer-motion": "^12.12.1",
  "fuse.js": "^7.1.0",
  "pg": "^8.16.3",
  "react": "^19.1.0",
  "react-dom": "^19.1.0"
},
```

Variables de entorno

Se creo un archivo en la raíz del proyecto con los datos de la conexión

```
🔧 .env
1  PGHOST=localhost
2  PGPORT=5432
3  PGDATABASE=condon
4  PGUSER=postgres
5  PGPASSWORD=1234
6
```

Módulo de conexión

Se implementó la conexión usando un **Pool de conexiones**

```
import 'dotenv/config';
import { Pool } from 'pg';
import type { QueryResult, QueryResultRow } from 'pg';

const num = (v?: string) => (v ? Number(v) : undefined);

export const pool = new Pool({
  host: process.env.PGHOST,
  port: num(process.env.PGPORT) ?? 5432,
  database: process.env.PGDATABASE,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  max: 10,
  idleTimeoutMillis: 30_000,
});

// 📌 nota el `extends QueryResultRow`
export async function query<T extends QueryResultRow = QueryResultRow>({
  text: string,
  params?: any[]
}): Promise<QueryResult<T>> {
  return pool.query<T>(text, params);
}
```

Conclusión

La base de datos “condon” constituye un modelo sólido para la gestión de un sistema de ventas. Permite controlar usuarios, productos, pedidos y pagos de manera eficiente, garantizando seguridad y escalabilidad.